

Taller Práctico: Asincronía y Callbacks en JavaScript

YEISON ALBERTO VARGAS SERANO

Objetivo

Implementar y analizar soluciones utilizando callbacks y tareas asíncronas en JavaScript para comprender cómo funciona el comportamiento no bloqueante del lenguaje.

Parte 1. Callbacks Básicos

Un callback es una función que se envía como argumento a otra función para ser ejecutada posteriormente.

Analiza el siguiente código y completa las secciones marcadas con TODO.

```
function realizarOperacion(num1, num2, operacionCallback) {
    console.log(`Operación con: ${num1} y ${num2}`);
    operacionCallback(num1, num2);
}

function sumar(a, b) {
    console.log(`Resultado Suma: ${a + b}`);
}

// TODO
function restar(a, b) {

}

// TODO
function multiplicar(a, b) {

}

realizarOperacion(10, 5, sumar);

// TODO
// Implementar las tres operaciones.
```

Actividades

1. Implementa el callback `restar`.
2. Implementa el callback `multiplicar`.
3. Ejecuta las tres operaciones.
4. Captura la salida obtenida en consola.

Preguntas de reflexión

1. ¿Qué ventaja tiene enviar una función como parámetro?

RTA: Permite reutilizar código y hacer funciones más flexibles, ya que la función principal puede ejecutar diferentes acciones sin necesidad de modificar su estructura.

2. ¿Qué ocurre si enviamos una función diferente como callback?

RTA: La función principal ejecutará esa nueva función, lo que cambia el comportamiento sin alterar el código base.

3. ¿La función principal necesita conocer qué operación matemática se ejecutará?

RTA: No, la función principal solo ejecuta el callback. No necesita saber qué operación específica se realiza.

Parte 2. Comprendiendo la Asincronía

Observa cuidadosamente el siguiente programa.

```
function tareaNoBloqueante(callback) {
  console.log("Iniciando tarea no bloqueante...");

  setTimeout(function() {
    console.log("Tarea completada.");
    callback();
  }, 2000);
}

console.log("Inicio del programa.");

tareaNoBloqueante(function() {
  console.log("Continuando después de la tarea.");
});

console.log("Fin del programa.");
```

Actividad

Antes de ejecutar el programa:

1. Escribe el orden que crees que aparecerá en la consola.
2. Ejecuta el código.
3. Registra el resultado real.

Análisis

Responde:

1. ¿Coincidió tu predicción con el resultado?

RTA: No, el orden fue diferente al que yo esperaba.

2. ¿Por qué el mensaje "Fin del programa" aparece antes que "Tarea completada"?

RTA: Porque JavaScript ejecuta el código de forma no bloqueante. La función `setTimeout` programa la ejecución del código interno después de 2 segundos, mientras que el resto del programa continúa ejecutándose sin esperar.

3. ¿Qué papel cumple `setTimeout()` en este comportamiento?

RTA: `setTimeout()` permite ejecutar una función después de un tiempo determinado sin detener la ejecución del programa principal. Esto demuestra el comportamiento asíncrono de JavaScript.

Parte 3. Simulación de Procesos Reales

Debes simular tres tareas comunes de una aplicación moderna.

Código Base

```
function solicitarJSON(callback) {  
  
}  
  
function reproducirAudio(callback) {  
  
}  
  
function leerSensor(callback) {
```

```
}

console.log("--- Iniciando procesos asíncronos ---");

solicitarJSON(() => console.log("Archivo JSON recibido.));
reproducirAudio(() => console.log("Audio finalizado.));
leerSensor(() => console.log("Datos del sensor listos.));
```

Requisitos

Implementa cada función utilizando `setTimeout()`.

Función	Tiempo
solicitarJSON	3000 ms
reproducirAudio	1000 ms
leerSensor	500 ms

Actividades

1. Implementa las tres funciones.
2. Ejecuta el programa.
3. Registra el orden real de finalización.

Resultado

Orden	Tarea
1	
2	
3	

Preguntas

1. ¿Cuál terminó primero?

RTA: La función `leerSensor`, porque tiene el menor tiempo (500 ms).

2. ¿Cuál terminó último?

RTA: La función `solicitarJSON`, porque tiene el mayor tiempo (3000 ms).

3. ¿Por qué el orden de finalización es diferente al orden de ejecución?

RTA: Porque las funciones se ejecutan de forma asíncrona. Aunque se llaman en un orden específico, cada una se completa según el tiempo definido en `setTimeout`, por lo que las tareas más rápidas terminan primero.

Parte 4. Diseñando tu Propia Operación Asíncrona

Reto

Crea una función llamada:

`simularDescarga(nombreArchivo, callback)`

La función deberá:

1. Mostrar un mensaje indicando que inició la descarga.
2. Esperar 4 segundos utilizando `setTimeout()`.
3. Mostrar un mensaje indicando que la descarga terminó.
4. Ejecutar el callback recibido.

Ejemplo esperado

```
simularDescarga("manual.pdf", () => {  
  console.log("Archivo listo para abrir.");  
});
```

Evidencia

Captura la salida de consola y explica qué sucede en cada paso.

RTA;

Explicación paso a paso

1. Se ejecuta la función `simularDescarga` con el archivo "manual.pdf".

2. Se muestra en consola el mensaje indicando que la descarga ha iniciado.
 3. La función `setTimeout` espera 4 segundos sin bloquear el programa.
 4. Después del tiempo, se muestra que la descarga ha finalizado.
 5. Finalmente, se ejecuta el callback mostrando que el archivo está listo para abrir.
-
-

Parte 5. Análisis del Callback Hell

Observa la siguiente estructura:

```
obtenerUsuario(() => {  
  obtenerPedidos(() => {  
    obtenerFactura(() => {  
      enviarCorreo(() => {  
        console.log("Proceso finalizado");  
      });  
    });  
  });  
});
```

Actividades

1. Describe el problema visual que presenta el código.
 2. Explica qué es el Callback Hell.
 3. ¿Por qué dificulta el mantenimiento de aplicaciones grandes?
 4. Investiga y menciona dos alternativas modernas para evitar este problema.
-

Entregables

El estudiante deberá entregar:

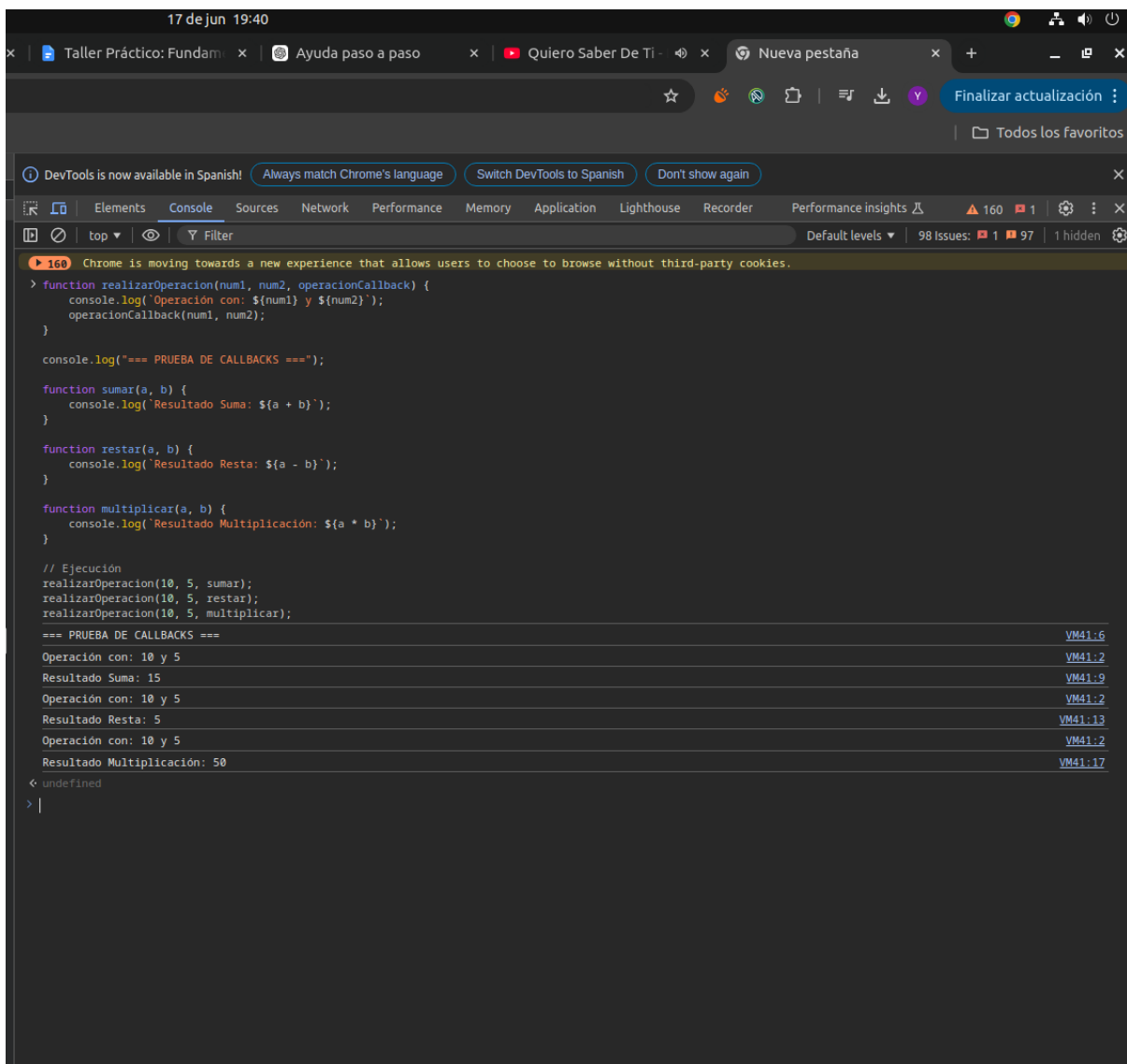
- Código fuente de todos los ejercicios.
- Capturas de consola de las ejecuciones.
- Respuestas a las preguntas de análisis.
- Explicación del ejercicio de Callback Hell.

RTA:

1. El código presenta una estructura muy anidada hacia la derecha, lo que lo hace difícil de leer y seguir.
2. El Callback Hell es una situación en la que múltiples funciones callback se anidan unas dentro de otras, generando un código difícil de leer, mantener y entender.
3. porque
 - Hace el código menos legible.
 - Complica la detección de errores.
 - Dificulta agregar nuevas funcionalidades.
 - Genera dependencia fuerte entre funciones.
- 4 . **Promesas (Promises):** Permiten manejar código asíncrono de forma más ordenada usando `.then()` y `.catch()`.
 - **Async/Await:** Es una forma más moderna que permite escribir código asíncrono como si fuera síncrono, mejorando mucho la legibilidad.

CAPTURAS DE PANTALLA

PARTE 1

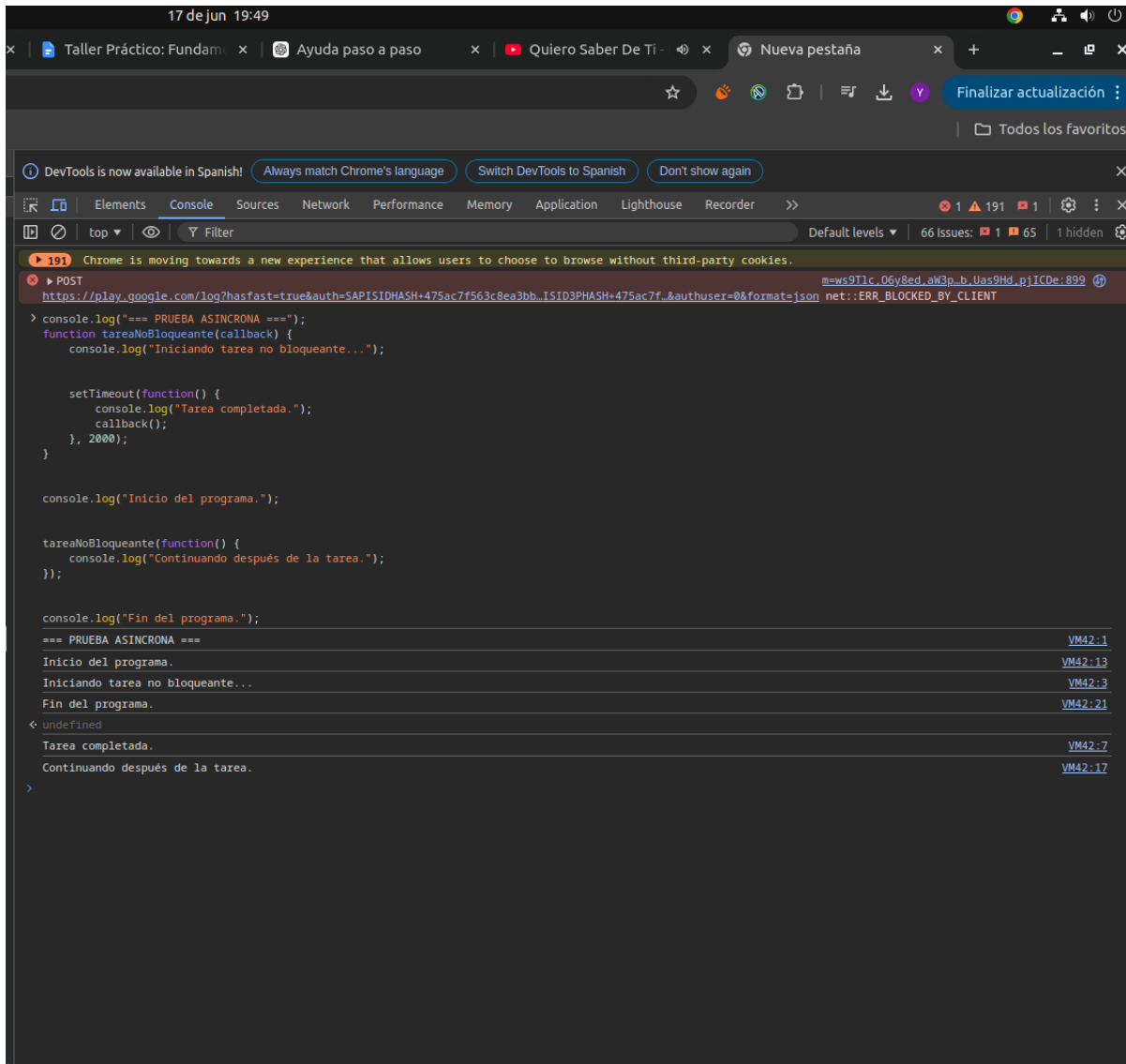


The screenshot shows a web browser window with several tabs. The active tab is titled "Nueva pestaña". Below the browser window, the Chrome DevTools interface is open, displaying the "Console" tab. The console shows a series of log messages and function definitions. The code defines a function `realizarOperacion` that takes three arguments: `num1`, `num2`, and `operacionCallback`. It logs the operation and then calls the callback function with the two numbers. Below this, there are three functions: `sumar`, `restar`, and `multiplicar`, each logging the result of an operation. The code then calls `realizarOperacion` with the values 10, 5, and each of the three functions. The console output shows the following messages:

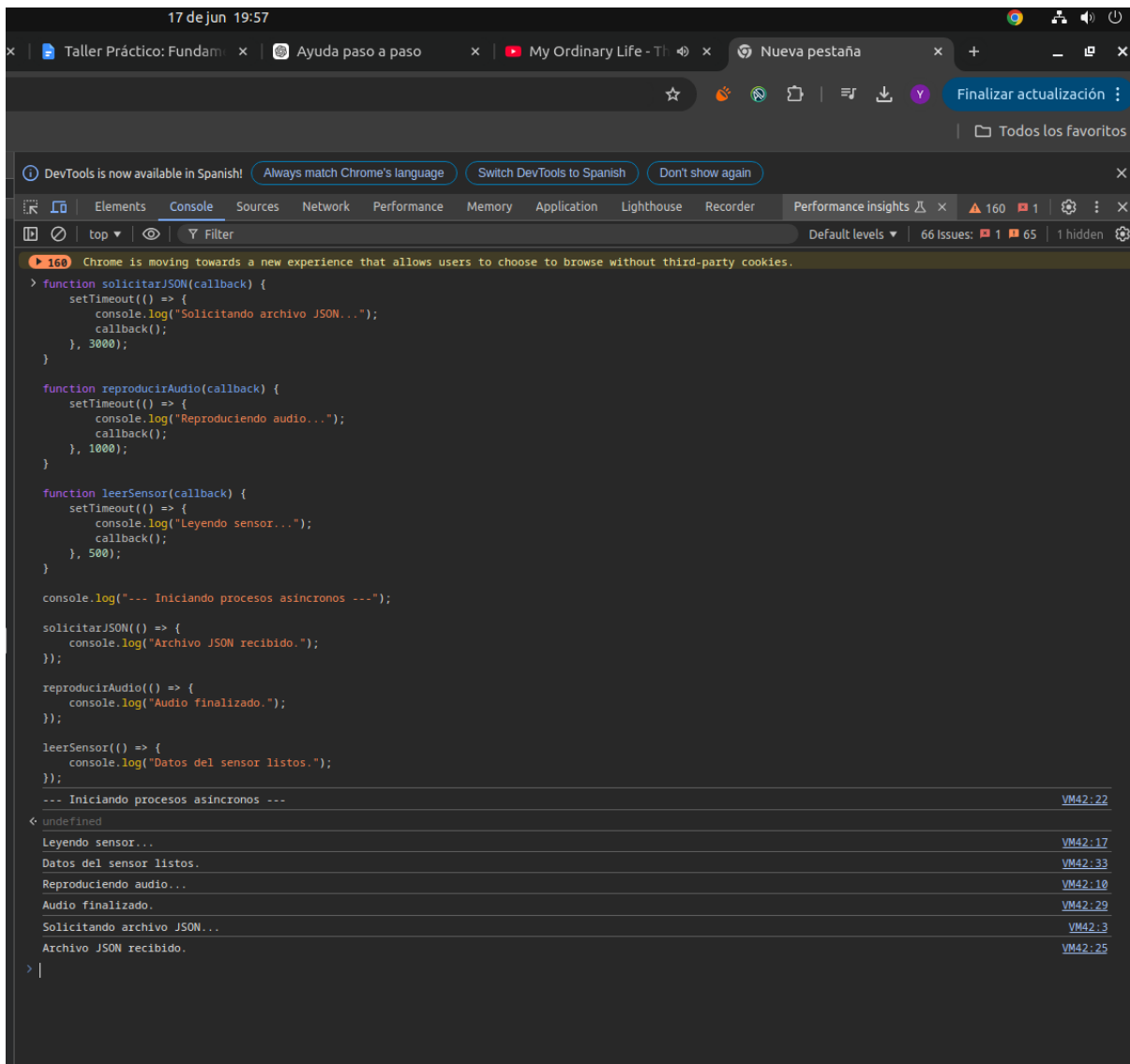
```
=== PRUEBA DE CALLBACKS ===
Operación con: 10 y 5
Resultado Suma: 15
Operación con: 10 y 5
Resultado Resta: 5
Operación con: 10 y 5
Resultado Multiplicación: 50
```

The console also shows a message from Chrome about moving towards a new experience that allows users to choose to browse without third-party cookies.

PARTE 2



PARTE 3



17 de jun 19:57

Taller Práctico: Fundam... x Ayuda paso a paso x My Ordinary Life - Th... x Nueva pestaña x

Finalizar actualización

Todos los favoritos

DevTools is now available in Spanish! Always match Chrome's language Switch DevTools to Spanish Don't show again

Elements Console Sources Network Performance Memory Application Lighthouse Recorder Performance insights 160 1 66 Issues: 1 65 1 hidden

top Filter

Chrome is moving towards a new experience that allows users to choose to browse without third-party cookies.

```
> function solicitarJSON(callback) {
  setTimeout(() => {
    console.log("Solicitando archivo JSON...");
    callback();
  }, 3000);
}

function reproducirAudio(callback) {
  setTimeout(() => {
    console.log("Reproduciendo audio...");
    callback();
  }, 1000);
}

function leerSensor(callback) {
  setTimeout(() => {
    console.log("Leyendo sensor...");
    callback();
  }, 500);
}

console.log("--- Iniciando procesos asincronos ---");

solicitarJSON(() => {
  console.log("Archivo JSON recibido.");
});

reproducirAudio(() => {
  console.log("Audio finalizado.");
});

leerSensor(() => {
  console.log("Datos del sensor listos.");
});

--- Iniciando procesos asincronos ---
VM42:22
< undefined
Leyendo sensor... VM42:17
Datos del sensor listos. VM42:33
Reproduciendo audio... VM42:10
Audio finalizado. VM42:29
Solicitando archivo JSON... VM42:3
Archivo JSON recibido. VM42:25
> |
```

PARTE 4

